C PROGRAMMING

DECISION MAKING & LOOP

1. Conditions – The Building Block

A condition is a question whose answer is TRUE or FALSE. In C, 0 is false and a non-zero value is true.

Operator	Meaning	Example
<	Less than	a < b
>	Greater than	a > b
<=	Less than or equal to	a <= b
>=	Greater than or equal to	a >= b
==	Equal to	a == b
!=	Not equal to	a != b

```
=   → assignment
==  → comparison
```

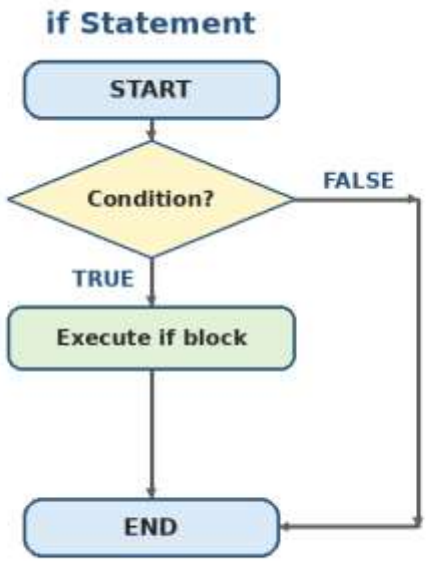
2. if Statement – One-Way Decision

The block executes only when the condition is TRUE.

SYNTAX	EXAMPLE

if (condition)
{
  statement;
}
	int age;
	scanf("%d", &age);
	if (age >= 18)
	  printf("Eligible to vote");

Flowchart



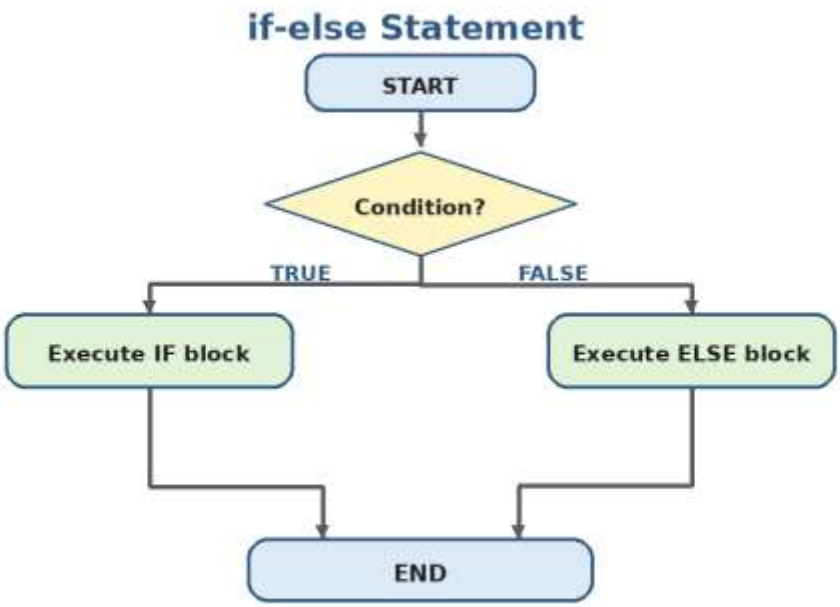
3. if-else Statement – Two Choices

The TRUE path executes the if block. The FALSE path executes the else block.

SYNTAX	EXAMPLE

if (condition)
{
  statement1;
}
else
{
  statement2;
}
	if (n % 2 == 0)
	  printf("Even");
	else
	  printf("Odd");

Flowchart



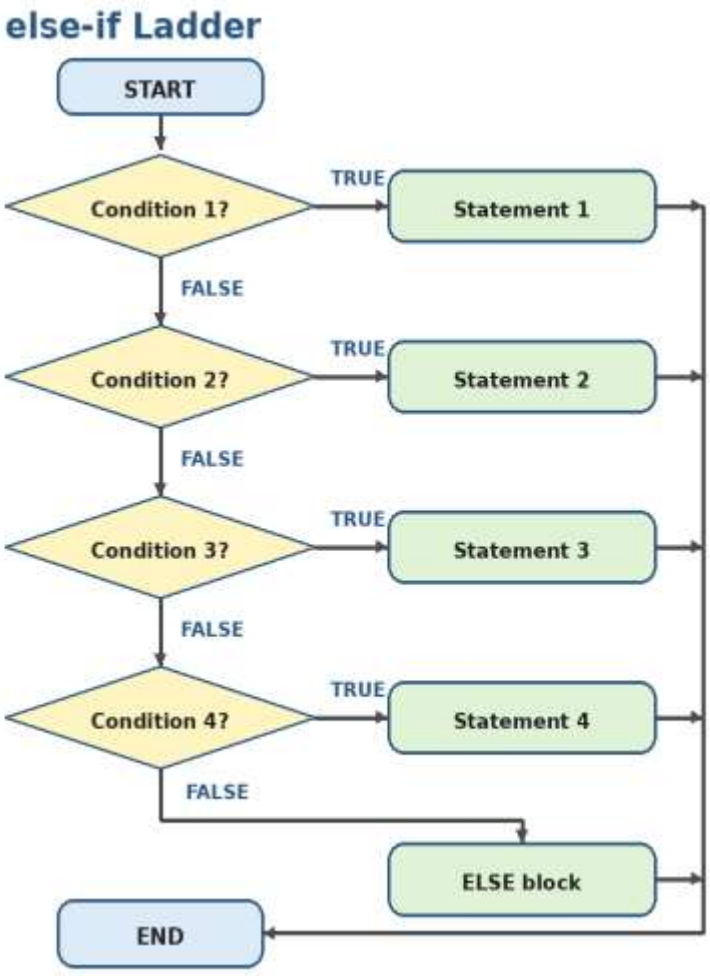
4. else-if Ladder – Multiple Conditions

Conditions are checked from top to bottom. The first TRUE condition is executed.

SYNTAX	EXAMPLE

if (condition1)
  statement1;
else if (condition2)
  statement2;
else if (condition3)
  statement3;
else
  statement4;
	if (marks >= 90) printf("A+");
	else if (marks >= 80) printf("A");
	else if (marks >= 70) printf("B");
	else if (marks >= 40) printf("Pass");
	else printf("Fail");

Flowchart



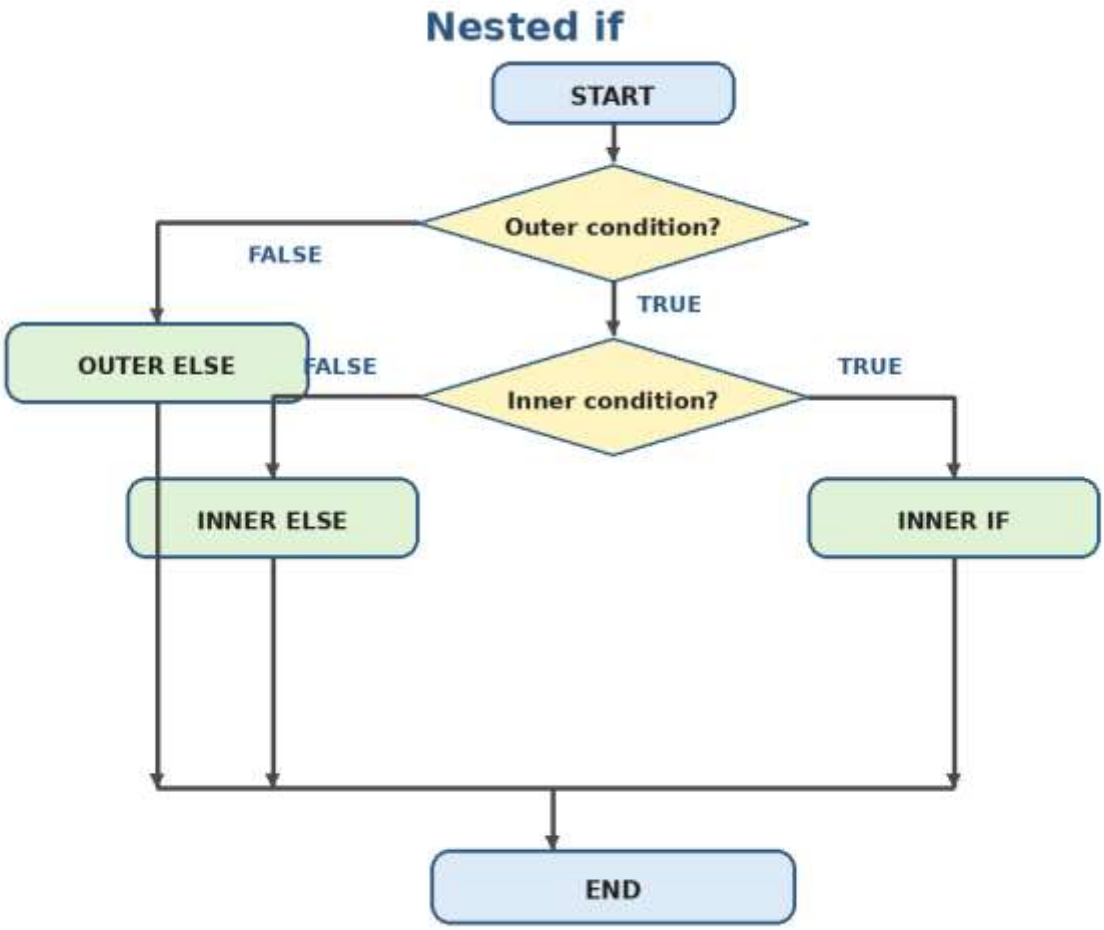
5. Nested if – Decision Inside a Decision

The inner condition is checked only when the outer condition is TRUE.

SYNTAX	EXAMPLE

if (condition1)
{
  if (condition2)
  {
  statement;
  }
}
	if (marks >= 40)
	{
	  if (marks >= 75)
	  printf("Distinction");
	  else
	  printf("Pass");
	}
	else
	  printf("Fail");

Flowchart



6. Ternary Operator (?:) – Short if-else

Use the ternary operator for a simple decision that produces one value.

SYNTAX	EXAMPLE

condition ? value_if_true : value_if_false;	int n = 8;
	char *result = (n % 2 == 0) ? "Even" : "Odd";
	printf("%s", result);

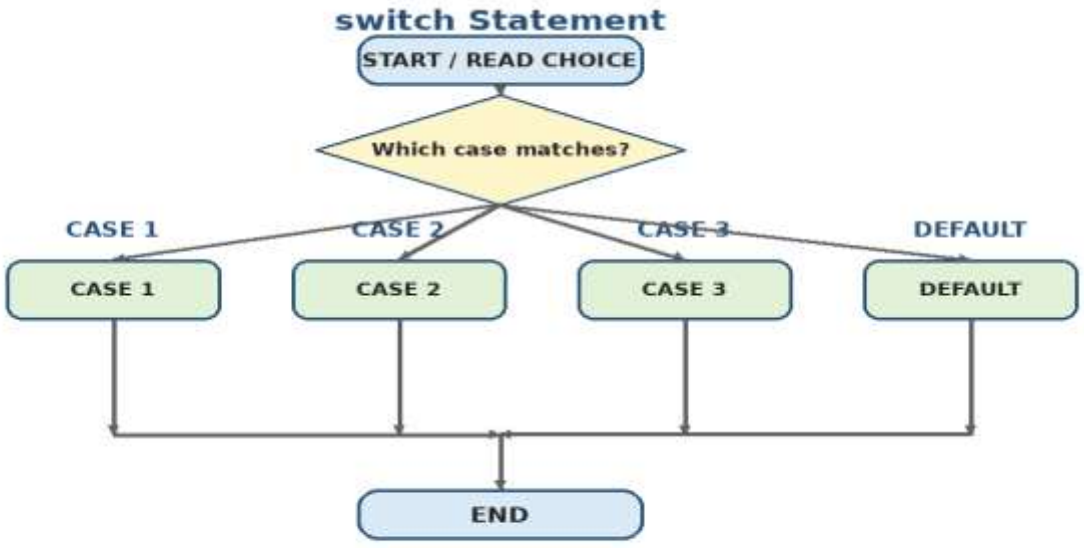
7. switch – Fixed Choices

Use switch when one expression is compared with several fixed values.

SYNTAX	EXAMPLE

switch (expression)
{
case value1:
statement;
break;
case value2:
statement;
break;
default:
statement;
}
	switch (choice)
	{
	case 1: printf("Add"); break;
	case 2: printf("Subtract"); break;
	case 3: printf("Multiply"); break;
	default: printf("Invalid choice");
	}

Flowchart



8. if-else vs switch

if-else	switch	Typical use
Can test ranges	Fixed values	Marks / grades vs menu
Complex conditions	Case values	Eligibility vs choices
Multiple conditions	Multiple cases	Range logic vs fixed options

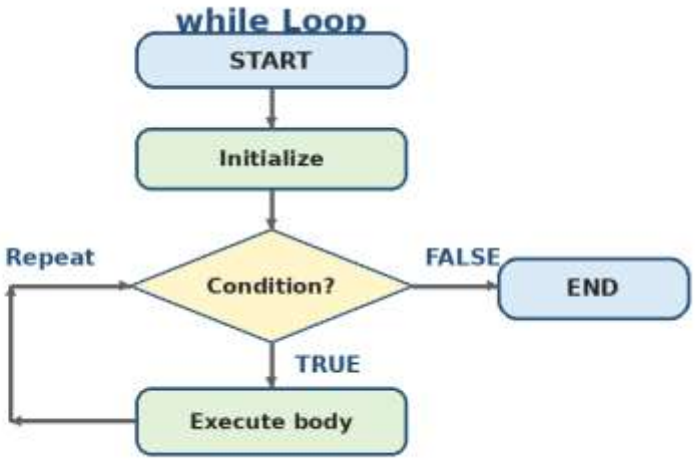
9. while Loop – Check First

The condition is checked before the body. Therefore, the loop can execute zero times.

SYNTAX	EXAMPLE

while (condition)
{
  statements;
}
	int i = 1;
	while (i <= 5)
	{
	  printf("%d ", i);
	  i++;
	}

Flowchart



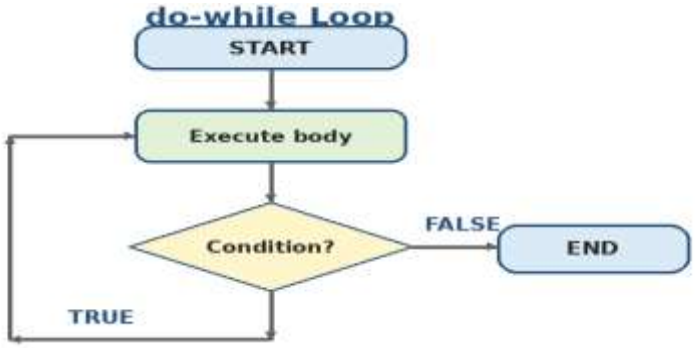
10. do-while Loop – Execute First

The body runs first and the condition is checked afterward. Therefore, it executes at least once.

SYNTAX	EXAMPLE

do
{
  statements;
}
while (condition);
	int i = 1;
	do
	{
	  printf("%d ", i);
	  i++;
	}
	while (i <= 5);

Flowchart



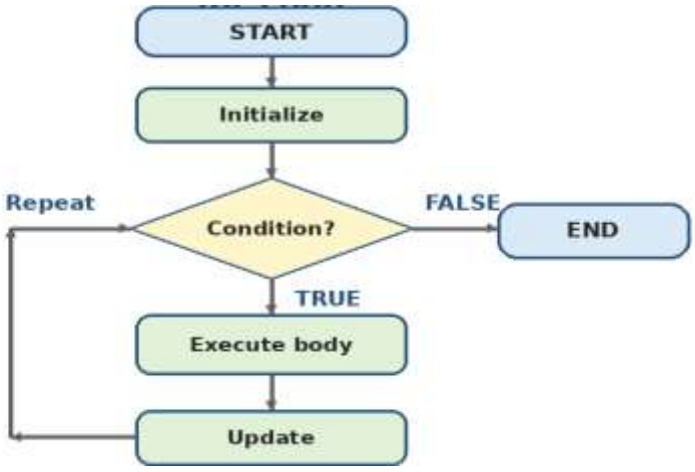
11. for Loop – Counter-Based

Use for when initialization, condition and update naturally belong together.

SYNTAX	EXAMPLE

for (initialization; condition; update)
{
  statements;
}
	for (int i = 1; i <= 5; i++)
	{
	  printf("%d ", i);
	}

Flowchart



12. Three Loops at a Glance

Loop	Condition checked	Minimum runs	Common use
while	Before body	0	Condition-controlled
do-while	After body	1	Menu / input
for	Before body	0	Counter / known repetitions

13. Nested Loops

A loop inside another loop. The inner loop completes its iterations for each iteration of the outer loop.

```
for (int i = 1; i <= 3; i++)
{
    for (int j = 1; j <= 2; j++)
    {
        printf("%d %d\n", i, j);
    }
}
```

14. break – Stop the Loop

break immediately exits the nearest loop.

```
for (int i = 1; i <= 10; i++)
{
    if (i == 5)
        break;
    printf("%d ", i);
}
// Output: 1 2 3 4
```

15. continue – Skip One Iteration

continue skips the remaining statements in the current iteration and moves to the next iteration.

```
for (int i = 1; i <= 5; i++)
{
    if (i == 3)
        continue;
    printf("%d ", i);
}
// Output: 1 2 4 5
```

16. Common Mistakes – if / else

- if (a = 5) → assignment. For comparison, use if (a == 5).
- if (a > 10); → the semicolon ends the if statement.
- Missing braces { } can make only the next statement part of the if.
- else belongs to the nearest unmatched if; use braces to make the pairing clear.
- In an else-if ladder, conditions are checked from top to bottom.
- Use && for AND and || for OR.

17. Common Mistakes – switch

- Do not forget break unless fall-through is intentional.
- Use constant values for case labels.
- Use default to handle an invalid choice.
- Do not use switch for range conditions such as marks >= 90.
- Do not repeat the same case value.

18. Common Mistakes – Loops

- Missing update → the loop may become infinite.
- Off-by-one error → check whether < or <= is required.
- Wrong update direction → i-- instead of i++ can stop the loop from reaching its condition.
- do-while requires a semicolon after while(condition);
- continue can cause an infinite loop if the loop-control variable is not updated.
- Use different variables such as i and j in nested loops.

19. Quick Revision

Concept	Meaning	Use
if	One-way decision	One condition
if-else	Two-way decision	Two outcomes
else-if	Multiple conditions	Grades / ranges
nested if	Decision inside a decision	Dependent conditions
?:	Short if-else	Simple value selection
switch	Fixed choices	Menu
while	Check first	Condition-controlled repetition
do-while	Execute first	At least one execution
for	Counter loop	Known repetitions
break	Exit	Stop early
continue	Skip iteration	Skip one turn